Privacy-Preserving RAG by Concealing Sensitive Information from External LLMs

Saleh Almohaimeed[1], Saad Almohaimeed[1], Mousa Jari[1], Fahad Alotaibi[1], Khalid A. Alobaid[1]

1. College of Applied Computer Science, King Saud University, Riyadh, Saudi Arabia

2. Department of Digital Transformation, Institution of Public Administration, Riyadh, Saudi Arabia

Abstract

Retrieval-Augmented Generation (RAG) is widely used to improve the performance of Large Language Models (LLMs) in answering user queries. Existing privacy research on RAG has focused on preventing unauthorized users from accessing sensitive data. However, another important problem that is often overlooked in RAG privacy research is that external generators have access to the query and the retrieved documents, which may contain confidential information that could potentially be misused or accessed for unintended purposes. In this paper, we introduce the Sensitive Entity Alias Generator (SEAG), a privacy-preserving framework that empowers users to utilize powerful third-party generators without disclosing sensitive information. SEAG introduces a lightweight model that locates sensitive entities, generates corresponding aliases, and constructs an entity replacement table. The table is used to replace sensitive words in the user's query and in the retrieved documents before they are forwarded to an external generator. For this purpose, two datasets were constructed: one for fine-tuning SEAG models to generate entity replacement tables, and another for evaluating the entire SEAG framework. The experimental results demonstrate the success of the SEAG framework. As for the User metric, which measures the ability of the model to provide a correct response to the user while hiding sensitive information from the external generator, all SEAG models achieved over 80% accuracy. Additional analysis further evaluated the ability of SEAG models Qwen-3, LLaMA-3.2, and Phi-4 to hide all sensitive entities within given documents. The results show good performance with total accuracies of 77.83%, 76.73%, and 74.91%, respectively.

Keywords: Retrieval-Augmented Generation (RAG); Privacy-Preserving AI; Large Language Models (LLMs); Sensitive Entity Replacement; Parameter-Efficient Fine-Tuning (PEFT)

1. Introduction

Retrieval-augmented generation (RAG) has been shown to improve the performance of Large Language Models (LLMs). Specifically, RAG enables LLMs to respond to user queries more effectively by combining LLM's internal knowledge with external knowledge derived from external data sources. The two main components of a RAG system are the retriever, which is responsible for retrieving relevant information, and the generator, which uses the information retrieved to generate accurate responses. This makes LLM a highly effective tool for company search, customer support, and question answering systems.

In spite of this, RAG experiences many privacy issues, which can be seen from two perspectives. First, user threat, as users may have access to sensitive information from external data sources for which they are not authorized. This issue has been widely discussed in many research papers [1][2][3]. Specifically, the user may deliberately or unintentionally submit questions that lead the RAG system to retrieve and reveal sensitive information from external data sources. Consequently, RAG generates responses that contain confidential data such as financial details [4], patient medical records [5], or contract information [6] that the user is not authorized to access.

The second privacy issue that has been overlooked by previous studies is the trustworthiness of the LLM generators. In some applications, the user may have full access to all data in external documents, such as when a physician queries their own patient records. However,

the external third-party generator (such as GPT-5 [7] or Claude-4 sonnet [8]) may still access and misuse the information. Several articles have reported this risk [9], [10], [11]. This threat can be mitigated by installing the generator locally, which ensures that sensitive data remain within the organization and are not exposed to third-party providers.

However, setting up a high-performance LLM locally can be very costly. For example, the cost of an NVIDIA H100 [12] GPU begins at approximately $25,000. The deployment of an LLM with more than 100 billion parameters typically requires several H100 GPUs. Furthermore, GPU costs are only one component of the overall cost of deployment. There are also other considerations, such as inference latency, number of concurrent users, networking equipment, and ongoing maintenance. This prevents many organizations from hosting high-performance LLM locally. Alternatively, small LLMs with parameters less than 30 billion can be deployed. However, the efficiency of the generator will be significantly reduced.

In this paper, we propose the Sensitive Entity Alias Generator (SEAG) framework, which enables organizations to leverage powerful external third-party large language models (LLMs), such as GPT-5 [7] and Claude-4 sonnet [8], while deploying only a lightweight local model with approximately 3–4 billion parameters. As illustrated in Figure 1, a traditional Retrieval-Augmented Generation (RAG) pipeline works as follows: when a user submits a query, the retriever identifies and returns the most relevant documents from external data sources. The query and the retrieved documents are then sent directly to the generator, which produces the final response.

In contrast, our SEAG framework introduces an additional component between the retriever and the generator. After retrieval, both the user query and the relevant documents are passed through the SEAG model, which creates an entity replacement table containing aliases for sensitive entities. Using this table, all sensitive entities in the query and retrieved documents are appropriately replaced with their corresponding aliases.

The anonymized query and documents are then forwarded to the external third-party generator. Since the SEAG model is fine-tuned to preserve semantic meaning and maintain consistency across the documents, the generator can process the information without being aware that sensitive entities have been replaced.

Once the generator produces a response, the output is passed through the entity replacement table once again. Any aliases appearing in the generated response are mapped back to their original entities, resulting in a final answer that preserves both the utility of the external LLM and the privacy of sensitive information.

Many methods can be used to hide sensitive words within a document. Nevertheless, in our work, we fine-tune a model to locate sensitive entities and generate a new alias for each entity while maintaining consistency. In order to accomplish this, we generate two datasets. One for fine-tuning the LLMs, and the other to evaluate the entire framework. The first dataset consists of 640 samples from 8 domains, and the second consists of 600 samples from 6 domains. The evaluation domains are different from those used during fine-tuning, except for two (Legal and Finance). With this setup, we can compare the performance of the fine-tuned models on domains it was trained on against domains not encountered during fine-tuning.

Three LLMs (Qwen-3 [13], LLaMA-3.2 [14], and Phi-4 [15]) have been finetuned using a parameter-efficient fine-tuning (PEFT) technique, called QLoRA [16], all of which are of size 3B with the exception of Qwen-3 [13], which is of 4B. The framework is then evaluated on two popular models, the GPT-5 [7] and Claude-4 sonnet [8]. Our SEAG framework is measured by two main metrics. The first metric is the Privacy metric, which measures whether the model correctly hides sensitive information by replacing them with meaningful words, and the results indicated that the best model was LLaMA-3.2 as a finetuned model, which scored 89.67%. The second metric is named User metric, which measures whether the whole SEAG framework has been implemented successfully. This means the sensitive information has been hidden and the correct results have been shown to the user. Based on this metric, the best results were achieved by the combination of LLaMA-3.2 as the SEAG model and Claude-4-sonnet as the generator, with a score of 83.67%.

Furthermore, we conducted a study to determine the performance of each fine-tuned SEAG model on hiding sensitive information, concluding that Qwen-3 [13] and LLaMA-3.2 [14] perform comparably, scoring 77.83% and 76.73%, respectively. Lastly, there are two major limitations that are demonstrated in the limitation section, including the length of the documents and the number of sensitive entities in the document.

Our main contributions are:

- To the best of our knowledge, we were the first to introduce a framework that protects private documents from being shared with the external third-party generator within RAG systems.
- Construct two datasets: The first for fine-tuning SEAG models in order to locate sensitive entities

within given documents and to generate semantically consistent aliases. The second is for evaluating the SEAG framework under three different metrics.

- Fine-tune three models, Qwen-3, LLaMA-3.2, and Phi-4 on our first dataset, and compare their performance in hiding sensitive information.
- Evaluation of the SEAG framework using two third-party generators GPT-5 and Claude-4 sonnet.

2. Related Work

In this section, we will present previous studies related to our work from three perspectives. First, the RAG application and its privacy issues. Second, the previous solutions done to identify sensitive entities within the text and the proposed solution. Third, the parameter-efficient fine-tuning of LLMs.

2.1. RAG application and its privacy issues

In recent years, RAG has been widely deployed in many organizations and private entities, as it can be applied to any application with some stored data, such as finance [4], medical [5], and legal applications [6].

There are many privacy issues associated with the RAG framework; the most common issue is the leakage of sensitive private data. In [1], [17], and [18], they propose different attack methods to extract sensitive information from databases. In [1] they craft a structured, well-designed prompt for attacking patients' medical records. In the HealthCareMagic [19] dataset, using 250 prompts, they were able to extract 89 medical dialogue chunks. In [17], they use an agent-based attack that generates adversarial prompts iteratively for extracting data from private RAG knowledge bases. In [18], they design a prompt-injected extraction attack, which tells the LLM to reproduce retrieved documents, enabling large-scale leakage of data. Furthermore, there are other attacks that do not use a structured prompt, such as membership inference attack [20][21][18], these attacks do not directly extract documents. Instead, they infer whether a specific record or document is present in the RAG knowledge base. Another attack is the Data Poisoning Attacks [22][23], these are knowledge corruption attacks on RAG, where an attacker could inject a few malicious texts into the knowledge database of a RAG system to induce an LLM to generate an attacker-chosen target answer for an attacker-chosen target question.

2.2. Sensitive Entity Identification

On the other hand, there are many efforts made to fight against these attacks and protect sensitive information in the retrieved documents within the RAG systems. A popular technique is using synthetic data [1][24][25]. In [24], they generate a privacy-preserving version of the original data and only provide the synthetic version to the LLM. They use a two-stage generation and refinement paradigm called SAGE, in stage-1 they utilize an attribute-based extraction and generation approach to generate the synthetic data. Then, they propose an agent-based iterative refinement approach to enhance the protection of private information. Their results demonstrate that using their generated synthetic data as RAG data achieves comparable performance to using the original data while effectively mitigating the associated privacy issues. Another solution is the application of the Differential Privacy technique [26][27], which protects sensitive information in the document by giving the LLM generator only a privacy-protected list of important concepts from the retrieved document. One major issue that has not been adequately addressed in previous work is the handling of sensitive documents in RAG systems. For example, a document may contain a patient's medical records, and an authorized user, such as a doctor, needs to query this information. However, we may not want the external generator LLM to directly access the sensitive content. In our work, we designed the SEAG framework to handle these issues while returning high-quality answers.

2.3. Parameter-efficient fine-tuning of LLM

After the tremendous adoption of RAG in many applications around the world, many researchers have explored improving RAG efficiency using Parameter-Efficient Fine-Tuning (PEFT) techniques, which improve domain adaptation while minimizing computational costs. Traditional fine-tuning approaches require updating all model parameters, which becomes prohibitively expensive for large language models (LLMs). PEFT methods, such as LoRA [28] and QLoRA [16], address this limitation by updating only a small subset of trainable parameters while keeping the original model weights frozen. The most common case is where they fine-tune the generator LLM in the RAG component in their own domain [29][30][31], as an example in [29], where they fine-tuned 5 different models on a healthcare dataset. The results reveal that all fine-tuned models outperform traditional RAG.

Another research direction focuses on fine-tuning the retriever component instead of the generator. The overall performance of a RAG system heavily depends on

the quality of retrieved documents. Improving the retriever can directly enhance answer accuracy. As a previous work in the OpenRAG paper [32], they fine-tune the retriever based on the downstream generation objective rather than relying solely on traditional retrieval relevance metrics. In our paper, we took a different approach where we did not focus on the retriever or the generator. To enhance privacy in RAG systems and achieve our objective, we fine-tune an LLM to act as a sensitive information entity recognition and replacement. The retriever will return the document, then it will pass through our fine-tuned LLM to hide sensitive information before the document is being used by the generator to get the answer.

3. Datasets

In the beginning, two datasets were constructed. First, we constructed a dataset to evaluate our main objective: fine-tuning LLMs to hide sensitive information in texts by replacing sensitive entities with alternatives that are semantically relevant. By doing so, we aim to prevent the reader LLM (Generator) from being aware that these entities have been substituted.

The second dataset was created for the purpose of evaluating the entire SEAG framework. First, the question and its associated documents are given to our fine-tuned SEAG model, that produces an entity replacement table. In both the question and the documents, sensitive entities are located and replaced based on the entity replacement table before being passed to the generator (GPT-5 [7] or Claude-4 sonnet [8]). Then, when the generator produces an answer, the entity replacement table is used again to return any substituted entities in the generated answer to their original values. Appendix A.3 contains a sample of entity replacement table.

The second dataset is used to assess the proposed SEAG framework from two perspectives. Firstly, the ability of the fine-tuned SEAG models to conceal sensitive information within the given text. Secondly, the ability of the external generator model to produce the correct answer based on the modified question and document.

3.1. Dataset for LLMs Fine-tuning

To construct the first dataset, we collected samples of documents from 8 different domains (news, legal, medical, finance, biology, chemistry, history, and resume). In such a way that every domain has 80 samples, resulting in 640 samples. Then, DeepSeek-V3 [33] is prompted to find entities within the given text and substitute them with semantically similar alternative words. We chose DeepSeek-V3 because it has demonstrated a good understanding of the Arabic language in comparison with other models in many research papers [34] [35].

A set of criteria is used to non-parametrically restrict DeepSeek-V3 [33] in the generation of the entity replacement table. There has been a strong emphasis on extracting all name entities, not just a few. Moreover, all replacement words should be consistent. For example, if there are two entities Saudi Arabia and Riyadh, which is the capital of Saudi Arabia. Then when DeepSeek-V3 replaces Saudi Arabia with Qatar, it must also replace Riyadh with the capital city of Qatar, which is Doha. Another criterion: if an entity appears in multiple forms, such as John, John Jordan, and John's, the entity replacement table must preserve those forms consistently. Create corresponding replacements for each variation, such as replacing John with Mike, John Jordan with Mike Cruz, and John's with Mike's. Lastly, the values in the original key must precisely and syntactically match the corresponding references in the document. For example, if a word is mentioned as John's, then the original key must have the exact word, not jhons, with small letters or John without an apostrophe and letter 's' at the end.

This set of criteria was not selected arbitrarily; we did this after so many refinements, first asking DeepSeek-V3 [33] to generate the entity replacement table, then checking for errors and repeating the process until we reached a stable situation where the generated entity replacement table was sufficient. Most importantly, we did not use a zero-shot prompt for DeepSeek-V3. We included an example question, document, and replacement table in the prompt as we realized that it would enhance the quality of the generated entity replacement table.

3.2. Dataset for evaluating the overall framework

To evaluate the overall SEAG framework, a second dataset was constructed from six different domains (economic, education, cars, legal, finance, business). Each domain contains 100 samples, resulting in a total of 600 samples. Two members of our team manually constructed the questions for this dataset. This was done to ensure that the question itself contains private entities and also asks about specific results in such a way that the answer should contain private information. The entities include numerical values, dates, names of objects, personal identifiers, etc. Additionally, the number of documents in the second dataset is 300, while the number of questions is 600, which means that there are two questions per document.

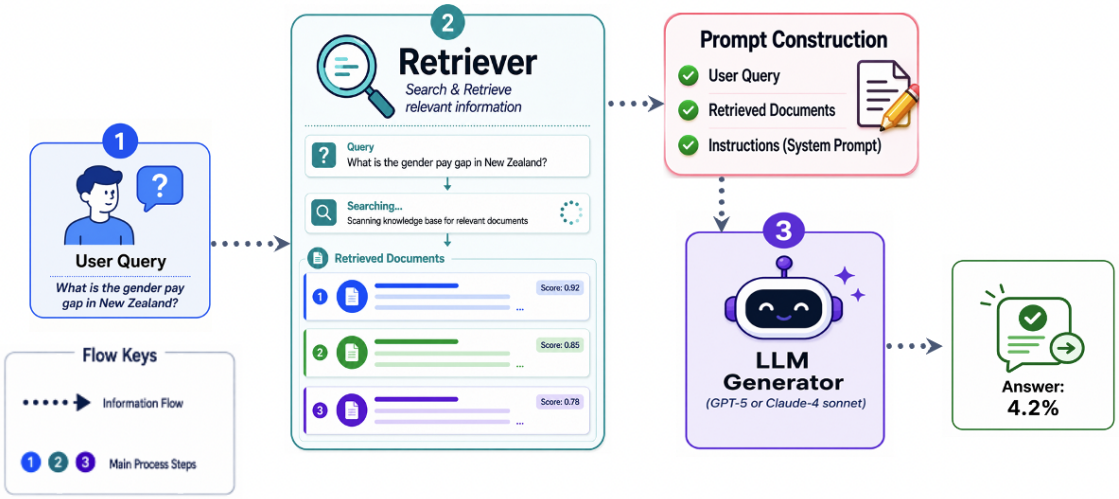

Figure 1: The diagram illustrates the structure of a traditional RAG system. It consists of three main components. First, user, who submits a natural language query to the system. Next, the system will use the retriever to extract the most relevant documents to the query from data sources. Following that, the RAG system will forward the query, relevant documents, and predefined system instructions to the generator in order to obtain the appropriate results.

This second dataset was not only built to evaluate the overall SEAG framework, but also to evaluate the performance of the fine-tuned models (SEAG models), to test their ability on hiding sensitive information. We have taken a considerable amount of time to identify each entity within the second dataset documents and evaluate whether the SEAG models were able to hide all of them successfully. The datasets and code are available on Github.com/Saleh-Almohaimeed/SEAG.

4. Methodology

4.1. Traditional RAG

Figure 1 shows the traditional Retrieval-Augmented Generation (RAG) system, including three main components: (1) User Query, (2) Retriever, and (3) Generator.

(Step 1), The user submits a query in natural language, for instance, "What is the gender pay gap in New Zealand?". The query is then passed to the Retrieval component (Step 2), which uses the existing knowledge base or document repository to retrieve the most relevant documents. The retriever assigns relevance scores to the retrieved documents then selects the top-k documents that are most likely to contain the information required to answer the user's query.

After that, the RAG system constructs a prompt based on three elements: user query, retrieved documents, and a set of system instructions that control the large language model behavior. The prompt provides additional information to the generator, allowing it to base its response on the retrieved information rather than relying completely on its internal knowledge.

In (Step 3) in Figure 1, the constructed prompt is forwarded to the generator LLM, such as GPT-5 [7] or Claude-4 sonnet [8], which integrates the retrieved information to produce the final answer. By incorporating external knowledge during inference, the RAG system mitigates hallucinations, increases factual accuracy, and enables the LLM to answer questions based on information that was not available during pre-training. In the example shown in Figure 1, the model correctly generates the answer "4.2%" based on the retrieved documents.

4.2. Sensitive Entity Alias Generator

The proposed SEAG framework is illustrated in Figure 2. As an extension of the traditional RAG pipeline, the framework utilizes a fine-tuned entity replacement model between prompt construction and the external LLM generator. The purpose of this model is to prevent sensitive information in the user query and retrieved documents from being exposed directly to the external generator, while still enabling the system to generate an accurate response. There are four main components in SEAG framework: (1) User Query, (2) Retriever, (3) SEAG model, and (4) Generator.

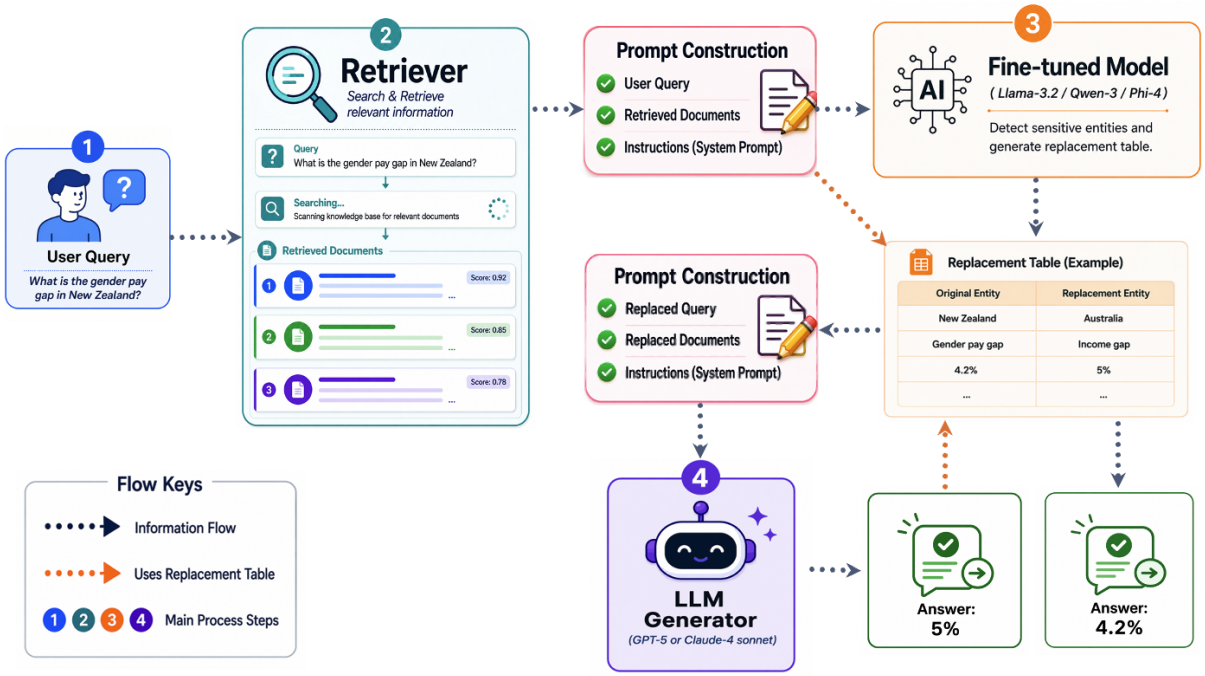

Figure 2: The structure of a traditional RAG system alongside our proposed SEAG framework. It consists of four main components. First, the user, who submits a natural language query to the system. Next, the system will use the retriever to extract the most relevant documents to the query from data sources. Following that, the SEAG framework will forward the query and relevant documents to the SEAG model, which will generate an entity replacement table. The table contains all sensitive entities within the query and documents, alongside appropriate aliases for these entities. After that, the SEAG framework will use this table to generate new versions of the query and documents that contain no real sensitive information and forward them to the generator. Finally, the generator will generate an answer, and if the answer contains sensitive entity values, using the entity replacement table, these values will be restored to their original values. Note: orange dotted arrows indicate the stages in which the entity replacement table is used.

Initially, the user submits a natural language query (Step 1). As the example shown in Figure 2, the user submits: "What is the gender pay gap in New Zealand?". The query is then sent to the retriever (Step 2), which will retrieve the most relevant documents from the data source. The documents retrieved are ranked according to their relevance scores and the top-k documents are chosen. (Steps 1 and 2) are the same as in traditional RAG. Step (3) of traditional RAG will take the original query, the retrieved documents, and predefined system instructions and pass them to the generator. However, the user query and retrieved documents may contain sensitive information that should not be shared with an externally hosted LLM (Generator). For instance, they may contain references to New Zealand, gender pay gap, and the value 4.2.

In the SEAG framework, in (Step 3) the original prompt is not sent directly to the generator. Instead, it is sent to a locally deployed fine-tuned model. The fine-tuned model is based on a relatively small language model, such as LLaMA-3.2 [14], Qwen-3 [13], or Phi-4 [15]. It is primarily responsible for identifying sensitive entities and generating appropriate replacements.

The fine-tuned model examines both the user's query and the retrieved documents to identify entities that need to be protected. After that, it creates an entity replacement table that maps every original entity with a corresponding substitute (alias). In the example, New Zealand is replaced with Australia, gender pay gap is replaced with income gap, and 4.2% is replaced with 5%. The entity replacement table is constructed in such a manner that the grammatical structure and semantic relationships within the query and documents are maintained.

The entity replacement table is important because it ensures that the same original entity is consistently replaced throughout the query as well as throughout all retrieved documents. As an example, every occurrence of New Zealand must be replaced with Australia, rather than using different country names in different parts of the prompt. In the same way, all related numerical values and other entities should be replaced consistently.

In this way, the external generator is able to reason over the transformed information without gaining access to the original sensitive information.

After the entity replacement table has been generated, it is applied to the original prompt. This produces a second, privacy-preserved prompt containing the replaced query, the replaced retrieved documents, and the original system instructions. In Figure 2, orange dotted arrows indicate the stages in which the entity replacement table is used.

In (Step 4), the replaced prompt is sent to an external third-party generator, such as GPT-5 [7] or Claude-4 sonnet [8]. The generator receives only the altered version of the information, therefore it does not have direct access to the original sensitive information. The generator then returns an intermediate result. In the illustrated example, it returns 5%, which is correct with respect to the replaced prompt.

However, this intermediate answer is not returned to the user directly. In the reverse direction, the entity replacement table is used again by the SEAG framework. Each replacement entity appearing in the generated answer is mapped back to its original counterpart. As a result, in Figure 2 the intermediate answer of 5% is restored to its original value of 4.2%. After the answer has been restored, it will be delivered to the user.

There is one main condition that must be met in order for the SEAG framework to be fully effective. The fine-tuned model must correctly identify and replace all sensitive entities consistently. A missing entity may expose private information, while an incorrect or inconsistent replacement may alter the meaning of the prompt.

5. Experiments

5.1. Experimental Setup

In order to ensure a fair comparison, all three SEAG models were fine-tuned using identical training hyperparameters. We used the AdamW optimizer with a learning rate of 5×10^{-4}, a batch size of 8, and trained each model for five epochs. It is true that model-specific hyperparameter tuning can potentially enhance individual performance, however, a fixed configuration was chosen in order to isolate the impact of the internal model architecture on the performance of SEAG model. A single NVIDIA T4 GPU was used for all experiments, and the Hugging Face transformers and PEFT libraries were used to implement the experiments.

5.2. Evaluation Metrics

Three different metrics have been used to evaluate the SEAG framework. Firstly, the **ORG** metric measures the ability of the generator (GPT-5 [7] or Claude-4 sonnet [8]) to produce the correct answer when given the original question and documents without using entity aliases. Our goal with this metric is to demonstrate that the questions and documents are not ambiguous and can be answered easily, which helps us identify whether our framework performs well. However, if the questions were difficult to answer, we were not able to determine from the other metrics (**Privacy** and **User**) whether the performance decrease was due to the difficulty of the questions or the inefficiency of the framework.

The second metric is **Privacy**, which measures whether the entities required for answering the question have been successfully hidden. As an example, if the ground-truth answer John cannot be answered by the generator LLM, then this is considered a successful hiding of the entity John. The third metric is **User**, which evaluates whether the correct answer is displayed to the user when the SEAG framework has been applied. It means that the ground truth answer is presented to the end user after it has been hidden from the external generator. For example, if the ground-truth answer is John, the generator answers with John, and the answer presented to the user is also John it is not considered correct. However, if the ground-truth answer is John, the generator answers with another name like Mike, and the answer presented to the user is John, then the answer is considered correct. In such a way John has been successfully replaced with another alias (Mike) and then restored back to John.

5.3. Overall Results

The results in Table 1 show that the generators GPT-5 and Claude-4 sonnet achieved very high ORG scores. ORG scores ranged from 99.33% to 99.67%, which indicates that the original questions and documents are clear and easy for the generators to answer without entity replacement. This confirms that the evaluation dataset questions are not difficult, and that any differences in performance with respect to Privacy and User metrics can be attributed more to the SEAG framework than to the difficulty of the questions.

According to the results of the Privacy metric, all three SEAG models were able to hide sensitive entities successfully in most cases. With Claude-4 sonnet as the generator, LLaMA-3.2 achieved the highest Privacy score of 89.67%, followed by Qwen-3 with 88.83%, and Phi-4 with 87.17%. Similar trends were observed with GPT-5, where LLaMA-3.2 achieved the highest Privacy score (88.50%), followed by Qwen-3 (87.50%) and Phi-4 (84.83%). The results indicate that LLaMA-3.2 produces the most effective entity replacements, making it

Table 1: Performance of different Sensitive Entity Alias Generator (SEAG) models with Claude-4 sonnet and GPT-5 as the generator. **ORG** measures the ability of the generator to produce the correct answer when given the original question and documents without any changes. **Privacy** measures whether the entities necessary for answering the question have been successfully hidden. **User** assesses whether the SEAG framework has been applied successfully and that the correct result is shown to the user while the generator sees the aliases instead. Note: The results of ORG metric are not related to the SEAG models, only to the generators.

Generator	SEAG Model	ORG	Privacy	User
Claude-4 sonnet	Qwen-3	99.33	88.83	83.00
	LLaMA-3.2	99.33	89.67	83.67
	Phi-4	99.33	87.17	81.17
GPT-5	Qwen-3	99.67	87.50	83.00
	LLaMA-3.2	99.67	88.50	83.00
	Phi-4	99.67	84.83	80.17

more difficult for external third-party generators to recover the original sensitive entities.

In the User metric, an evaluation is made of whether the correct final answer is returned to the user following the restoration of the hidden entities. With Claude-4 sonnet, LLaMA-3.2 achieved the highest User score (83.67%), followed closely by Qwen-3 (83.00%), while Phi-4 obtained 81.17%. In GPT-5, both Qwen-3 and LLaMA-3.2 achieved 83.00%, whereas Phi-4 achieved 80.17%. Based on these results, it can be concluded that the SEAG framework is capable of maintaining quality of answers for the end user, while protecting sensitive entities from external third-party generator.

Overall, LLaMA-3.2 consistently provided the highest Privacy scores while maintaining high User accuracy across both generators. Furthermore, similar results obtained with Claude-4 sonnet and GPT-5 demonstrate the effectiveness of the SEAG framework across a variety of state-of-the-art generators. Since the Privacy metric evaluates only those entities necessary to answer the user's question, we evaluated all three SEAG models on their capability to hide all named entities in the retrieved documents. The results of this more comprehensive evaluation are presented in the following section.

5.4. Hiding Sensitive Information

Table 2 illustrates the entity replacement accuracy across six domains for the SEAG models. Overall, Qwen-3 had the highest overall accuracy (77.83%), followed by LLaMA-3.2 (76.73%) and Phi-4 (74.91%). Since each of the six domains contains 100 samples with approximately 15 sensitive entities per sample, the reported domain accuracy reflects the proportion of correctly replaced sensitive entities in that domain. Interestingly, despite the fact that the SEAG models were fine-tuned using financial and legal data, neither of these domains achieved the highest performance. In contrast, economic consistently produced the best results for all three models, with accuracy ranging from 86.01% to 87.46%.

Table 2: Entity replacement accuracy across different domains for the evaluated SEAG models. The **Total Accuracy** is computed over all six domains (600 samples in total), while each domain score is calculated over the entities belonging to that domain.

Domain	Qwen-3	Phi-4	LLaMA-3.2
Total Accuracy	77.83	74.91	76.73
Economic	86.01	86.11	87.46
Education	75.45	71.01	74.72
Cars	75.14	74.94	76.16
Legal	76.33	73.76	75.27
Finance	77.49	73.10	74.13
Business	77.70	72.13	74.69

When we have manually examined the dataset, it appears that the economic domain is dominated by factual entities whose boundaries are explicit and whose semantic categories are clearly identifiable. Examples include countries (New Zealand, United States), international organizations (OECD, IMF), dates and years (April 2025), percentages (10%, 15%), monetary amounts (USD 900 billion), and well-defined programs such as NextGenerationEU. As these entities usually appear as single nouns, phrases or numerical expressions with little ambiguity, SEAG models can accurately identify and replace them while preserving their surrounding context.

However, the legal and finance domains contain entities that are more context-specific and structurally complex. Legal documents frequently include long legal or policy titles, and hierarchical references. (e.g., Medium-Term Fiscal-Structural Plan 2025–2029 (MTFSP), and National Recovery and Resilience Plan (NRRP)), where accurately determining the complete entity span is more challenging than replacing a standalone country or year.

Similarly, finance documents contain institution-specific organizations, financial reports, and monetary expressions that are often used together within the same statement, such as McKinsey, PWC, S&P Global, CFA Institute. Thus, in order to perform a correct replacement, it is necessary to simultaneously recognize multiple related entities while maintaining their semantic relationships. Therefore, the superior performance on the economic domain is likely due to its use of standardized, explicitly outlined entities rather than any advantages resulting from domain-specific fine-tuning.

6. Limitations

6.1. Document Length and Entity Density

Our work results are based on documents with an average length of approximately 300 words, each containing approximately 15 sensitive entities. This size is representative of many practical RAG applications, in which the retrieved chunks are often concise to fit within the model context while still providing sufficient information to answer user questions. However, the proposed framework has not been evaluated on documents that are substantially longer and contain a large number of entities. As document length increases, identifying, replacing, and consistently restoring all sensitive entities may become more difficult, potentially affecting both privacy preservation and the quality of answers. For this reason, future work should explore the scalability of the SEAG framework on larger documents with multiple sensitive entities in order to gain a better understanding of its robustness in more challenging situations.

7. Conclusion

In this paper, we present the Sensitive Entity Alias Generator (SEAG), a privacy-preserving framework for RAG systems. Unlike previous approaches, in which the primary objective is to prevent unauthorized users from accessing sensitive information, the SEAG framework focuses on preventing external third-party generators from accessing sensitive information. In order to accomplish this, the SEAG model is locally deployed between the retriever and the generator, where it detects sensitive entities within the query and documents. Then, generate semantically consistent aliases for these entities. This will ensure that the external third-party generator will only see the aliases and not the original sensitive entities.

To evaluate the proposed framework we constructed two datasets: one to fine-tune a lightweight LLM for finding sensitive entities in given documents and the other to evaluate the entire SEAG framework across multiple domains.

Experimental results show that the proposed framework successfully hides sensitive information while maintaining high answer quality. In comparison to other models, LLaMA-3.2 achieved the best overall performance, achieving the highest Privacy score and maintaining a high User accuracy across both GPT-5 and Claude-4 sonnet generators.

Also, another experiment was conducted to evaluate each SEAG model across all entities in the retrieved documents. Results indicate that LLaMA-3.2 and Qwen-3 achieve very close performance, with a slight advantage being given to Qwen-3. However, Phi-4 performance was clearly less than the others as it achieved 2% lower results across all six domains.

Despite the promising results of the proposed framework, several challenges remain. Future research should examine longer documents, documents containing substantially larger numbers of sensitive entities, additional categories of sensitive information, multilingual settings, as well as stronger replacement strategies capable of handling more complex contextual dependencies. Taking these actions will improve the scalability and robustness of RAG systems.

References

[1] S. Zeng, J. Zhang, P. He, Y. Xing, Y. Liu, H. Xu, J. Ren, S. Wang, D. Yin, Y. Chang, J. Tang, The good and the bad: Exploring privacy issues in retrieval-augmented generation (rag), in: Findings of the Association for Computational Linguistics: ACL 2024, Association for Computational Linguistics, Bangkok, Thailand, 2024, pp. 4505–4524. doi:10.18653/v1/2024.findings-acl.267.

[2] Y. Zhou, W. Zhang, J. Shao, Y. Liu, X. Li, J. Jin, H. Qian, Z. Liu, C. Li, J. C. Zhang, Z. Dou, P. S. Yu, J. Mao, Trustworthiness in retrieval-augmented generation systems: A survey, arXiv

preprint arXiv:2409.10102 (2024). doi:10.48550/arXiv.2409.10102.

[3] Y. Zhang, J. Wu, R. Li, T. Zhang, Y. Song, C. Li, S. Wang, H. Shen, J. Yin, J. Ge, B. Luo, Privacy protection in rag: A novel method and evaluation framework, Information Processing & Management 63 (2026) 104505. doi:10.1016/j.ipm.2025.104505.

[4] R. Nagpal, U. Usua, R. Palacios, A. Gupta, FairRAG: A privacy-preserving framework for fair financial decision-making, Applied Sciences 15 (15) (2025) 8282. doi:10.3390/app15158282.

[5] T. B. Weerasekara, C. Chandeepa, O. S. Amarasuriya, C. Hettiarachchi, Privacy-preserving medical advising system on mobile devices: On-device phi anonymization, medical report retrieval, and cloud-based rag, in: Proceedings of the ACM/IEEE International Conference on Connected Health: Applications, Systems and Engineering Technologies, Association for Computing Machinery, 2025, pp. 447–452. doi:10.1145/3721201.3725431.

[6] M. Hindi, L. Mohammed, O. Maaz, A. Alwarafy, Enhancing the precision and interpretability of retrieval-augmented generation (rag) in legal technology: A survey, IEEE Access 13 (2025) 46171–46189. doi:10.1109/ACCESS.2025.3550145.

[7] OpenAI, GPT-5 system card, OpenAI, accessed: 12 August 2026 (Aug. 2025).
URL https://openai.com/index/gpt-5-system-card/

[8] Anthropic, Introducing Claude 4, Anthropic, accessed: 12 August 2026 (May 2025).
URL https://www.anthropic.com/news/claude-4

[9] N. G. Itoi, Be careful what you tell your AI chatbot, Stanford Institute for Human-Centered Artificial Intelligence, accessed: 12 August 2026 (Oct. 2025).
URL https://hai.stanford.edu/news/be-careful-what-you-tell-your-ai-chatbot

[10] Financial Times, Florida sues OpenAI and sam altman for "hurting" children, Financial Times, accessed: 12 August 2026 (Jun. 2026).
URL https://www.ft.com/content/7998db0a-22d1-4b80-b861-2e8b2448e97f

[11] E. Pollina, A. Armellini, Italy fines OpenAI over ChatGPT privacy rules breach, Reuters, accessed: 12 August 2026 (dec 2024).
URL https://www.reuters.com/technology/italy-fines-openai-15-million-euros-over-privacy-rules-breach-2024-12-20/

[12] NVIDIA Corporation, NVIDIA H100 GPU, NVIDIA, accessed: 12 August 2026.
URL https://www.nvidia.com/en-us/data-center/h100/

[13] A. Yang, A. Li, Yang, Qwen3 technical report, arXiv preprint arXiv:2505.09388 (2025). doi:10.48550/arXiv.2505.09388.
URL https://arxiv.org/abs/2505.09388

[14] Meta AI, Llama 3.2 model card, accessed: 12 August 2026 (2024).
URL https://github.com/meta-llama/llama-models/blob/main/models/llama3_2/MODEL_CARD.md

[15] A. Abouelenin, A. Ashfaq, A. Atkinson, H. Awadalla, N. Bach, J. Bao, A. Benhaim, M. Cai, V. Chaudhary, C. Chen, et al., Phi-4-Mini technical report: Compact yet powerful multimodal language models via mixture-of-LoRAs, arXiv preprint arXiv:2503.01743 (2025). doi:10.48550/arXiv.2503.01743.
URL https://arxiv.org/abs/2503.01743

[16] T. Dettmers, A. Pagnoni, A. Holtzman, L. Zettlemoyer, QLoRA: Efficient finetuning of quantized LLMs, in: Advances in Neural Information Processing Systems, Vol. 36, Curran Associates, Inc., 2023, pp. 10088–10115. doi:10.52202/075280-0441.
URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/1feb87871436031bdc0f2beaa62a049b-Abstract-Conference.html

[17] C. Jiang, X. Pan, G. Hong, C. Bao, Y. Chen, M. Yang, Feedback-guided extraction of knowledge base from retrieval-augmented LLM applications, arXiv preprint arXiv:2411.14110 (2024). doi:10.48550/arXiv.2411.14110.

[18] Z. Qi, H. Zhang, E. P. Xing, S. M. Kakade, H. Lakkaraju, Follow my instruction and spill the

beans: Scalable data extraction from retrieval-augmented generation systems, in: The Thirteenth International Conference on Learning Representations, International Conference on Learning Representations, 2025.
URL https://openreview.net/forum?id=Y4aWwRh25b

[19] Y. Li, Z. Li, K. Zhang, R. Dan, S. Jiang, Y. Zhang, ChatDoctor: A medical chat model fine-tuned on a large language model meta-ai (LLaMA) using medical domain knowledge, GitHub repository, accessed: 12 August 2026 (2023).
URL https://github.com/Kent0n-Li/ChatDoctor

[20] G. Wang, J. He, H. Li, M. Zhang, D. Feng, RAG-leaks: Difficulty-calibrated membership inference attacks on retrieval-augmented generation, Science China Information Sciences 68 (6) (2025) 160102. doi:10.1007/s11432-024-4441-4.

[21] M. Liu, S. Zhang, C. Long, Mask-based membership inference attacks for retrieval-augmented generation, in: Proceedings of the ACM Web Conference 2025, WWW '25, Association for Computing Machinery, Sydney, NSW, Australia, 2025, pp. 2894–2907. doi:10.1145/3696410.3714771.

[22] W. Zou, R. Geng, B. Wang, J. Jia, PoisonedRAG: Knowledge corruption attacks to Retrieval-Augmented generation of large language models, in: 34th USENIX Security Symposium (USENIX Security 25), USENIX Association, Seattle, WA, 2025, pp. 3827–3844.
URL https://www.usenix.org/conference/usenixsecurity25/presentation/zou-poisonedrag

[23] H. Chaudhari, G. Severi, J. Abascal, A. Suri, M. Jagielski, C. A. Choquette-Choo, M. Nasr, C. Nita-Rotaru, A. Oprea, Phantom: General backdoor attacks on retrieval augmented language generation, ACM Transactions on AI Security and Privacy (Mar. 2026). doi:10.1145/3796729.
URL https://doi.org/10.1145/3796729

[24] S. Zeng, J. Zhang, P. He, J. Ren, T. Zheng, H. Lu, H. Xu, H. Liu, Y. Xing, J. Tang, Mitigating the privacy issues in retrieval-augmented generation (RAG) via pure synthetic data, in: Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, Association for Computational Linguistics, Suzhou, China, 2025, pp. 24538–24569. doi:10.18653/v1/2025.emnlp-main.1247.
URL https://aclanthology.org/2025.emnlp-main.1247/

[25] Y. Yu, Y. Zhuang, J. Zhang, Y. Meng, A. J. Ratner, R. Krishna, J. Shen, C. Zhang, Large language model as attributed training data generator: A tale of diversity and bias, in: Advances in Neural Information Processing Systems, Vol. 36, Curran Associates, Inc., 2023. doi:10.52202/075280-2433.
URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/ae9500c4f5607caf2eff033c67daa9d7-Abstract-Datasets_and_Benchmarks.html

[26] T. Tang, J. Flemings, Y. Wang, M. Annavaram, Differentially private retrieval-augmented generation, arXiv preprint arXiv:2602.14374 (Feb. 2026). doi:10.48550/arXiv.2602.14374.
URL https://arxiv.org/abs/2602.14374

[27] N. Grislain, RAG with differential privacy, in: 2025 IEEE Conference on Artificial Intelligence (CAI), IEEE, 2025, pp. 847–852.
URL https://ieeexplore.ieee.org/document/11050672

[28] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen, LoRA: Low-rank adaptation of large language models, in: International Conference on Learning Representations, 2022.
URL https://openreview.net/forum?id=nZeVKeeFYf9

[29] B. Pingua, A. Sahoo, M. Kandpal, D. Murmu, J. Rautaray, R. K. Barik, M. J. Saikia, Medical LLMs: Fine-tuning vs. retrieval-augmented generation, Bioengineering 12 (7) (2025) 687. doi:10.3390/bioengineering12070687.
URL https://doi.org/10.3390/bioengineering12070687

[30] P. Devine, ALoFTRAG: Automatic local fine tuning for retrieval augmented generation, arXiv preprint arXiv:2501.11929 (Jan. 2025). doi:10.48550/arXiv.2501.11929.
URL https://arxiv.org/abs/2501.11929

[31] Y. He, X. Zhu, D. Li, H. Wang, Enhancing large language models for specialized domains: A two-stage framework with parameter-sensitive LoRA

fine-tuning and chain-of-thought RAG, Electronics 14 (10) (2025) 1961. doi:10.3390/electronics14101961.
URL https://doi.org/10.3390/electronics14101961

[32] S. B. Islam, M. A. Rahman, K. S. M. T. Hossain, E. Hoque, S. Joty, M. R. Parvez, OpenRAG: Enhanced retrieval augmented reasoning with open-source large language models, in: Findings of the Association for Computational Linguistics: EMNLP 2024, Association for Computational Linguistics, Miami, Florida, USA, 2024, pp. 14231–14244. doi:10.18653/v1/2024.findings-emnlp.831.
URL https://aclanthology.org/2024.findings-emnlp.831/

[33] DeepSeek-AI, DeepSeek-V3 technical report, arXiv preprint arXiv:2412.19437 (2024). doi:10.48550/arXiv.2412.19437.
URL https://arxiv.org/abs/2412.19437

[34] S. Almohaimeed, S. Almohaimeed, M. Jari, K. A. Alobaid, F. Alotaibi, AI text detectors and the misclassification of slightly polished arabic text, Journal of Big Data (Jun. 2026). doi:10.1186/s40537-026-01492-8.
URL https://doi.org/10.1186/s40537-026-01492-8

[35] S. Almohaimeed, A. Alabduljabbar, M. Jari, M. Alkhowaiter, S. Almohaimeed, M. M. Al Rahhal, Benchmarking retrieval augmented generation LLMs for arabic noise robustness, Frontiers in Big Data 9 (2026). doi:10.3389/fdata.2026.1884673.
URL https://doi.org/10.3389/fdata.2026.1884673

Appendix A. Example of Replacement Table

Table A.3 shows an example of an entity replacement table generated by Phi-4 model. The table has the detected sensitive entities, their corresponding entity types, and the generated aliases used to anonymize the user query and retrieved documents before they are sent to the external LLM.

Table A.3: Example of a replacement table generated by the fine-tuned Phi-4 model.

Entity Type	**Original Entity**	**Replacement Entity**
Date	April	March
Location	New Zealand	Australia
Date	2025	2026
Organization	OECD	EU
Figure	Figure 1.1	Figure 2.2
Location	Middle East	South Asia
Location	United States	Canada
Percentage	10%	8%
Percentage	15%	12%
Date	August	July
Date	April 2026	March 2027
Date	2026	2027
Commodity	dairy	grain
Commodity	beef	meat
Sector	Tourism	Manufacturing